

差旅出行助手Agent(面试准备)

自我介绍

面试官您好，我叫xxx，本科毕业于xx大学，硕士就读于xx大学xx学院，在校期间我曾多次获得过奖学金，我的研究方向是xx，在校期间我积极参与科研项目，我做过的项目有xxx、xxx。此外，我还有一段实习经历，是在xxx公司做Agent开发实习生，主要参与的项目是差旅出行助手Agent，是基于AgentScope框架的多Agent系统，可以给用户提供个性化的行程规划服务，我参与了整体架构设计和核心模块的实现。

以上是我的自我介绍，需要我给您详细介绍一下这个Agent项目吗？

简历写法

差旅出行助手Agent | [公司名称] 大模型开发工程师/实习生

2026.02-2026.05

项目背景

传统旅行规划方式效率低、体验差，用户需要在多个平台查询景点、攻略、酒店、交通。初版Agent系统采用串行调度和关键词匹配，意图识别准确率仅65%，响应时间达30秒，无法支持用户偏好记忆和企业知识查询。

解决方案

- 两层记忆系统**：设计短期记忆+长期记忆+异步LLM总结机制，实现PreferenceAgent智能识别追加/覆盖偏好动作；引入Redis缓存层（用户偏好热数据、LLM总结结果），缓存命中率85%；
- RAG知识库**：基于Milvus向量数据库+BGE-m3 Embedding模型+滑动窗口Chunking策略，构建差旅政策、城市攻略、FAQ知识库，采用余弦相似度检索+文档溯源保证可信度；集成Google Search API实现实时信息查询（天气、限行等）；
- 多Agent协作优化**：重构IntentionAgent为基于LLM语义理解的多意图识别系统（支持6大类意图），实现"优先级+并行"混合调度模式，响应时间从30秒降至15秒；
- Skill Plugins插件化架构**：实现LazyAgentRegistry动态发现机制+懒加载（系统启动时仅扫描不加载，首次调用时按需加载并缓存），支持灵活增减Agent无需修改核心代码；采用Progressive Disclosure渐进式暴露机制（意图识别阶段仅加载Skill元数据，执行阶段按需加载详细指令），系统启动速度优化至3秒。

项目成果

- 核心指标提升**：意图识别准确率从65%提升至90%+，知识库问答准确率95%，用户偏好记忆准确率95%，系统响应时间从30秒降至15秒；

- 系统能力完善：支持意图识别、事项收集、偏好管理、知识问答、信息查询、行程生成6大功能模块。

项目描述

先介绍项目背景，再说一开始是怎么做的，但是遇到了什么困难、挑战，尝试了什么新的方法，结论是什么（数值型评估指标），一定要突出项目“迭代”的过程。

这个项目的目标是开发一个智能旅行规划助手。公司希望利用大模型技术给用户提供个性化的行程规划服务，我负责的是整个多Agent系统的架构设计和核心模块实现。之所以要做这个系统，是因为传统的旅行规划方式效率很低、体验也不好。很多用户想规划一次旅行，得在不同平台上查景点、看攻略、订酒店、查交通，至少要花2-3个小时，而且系统完全不记得用户上次去哪玩过、喜欢住什么酒店这些偏好信息，每次都要重新输入，非常麻烦。

最初尝试用简单的Agent架构来解决这个问题。我们基于AgentScope框架实现了意图识别和行程规划两个基础Agent，采用了串行调度的方式。具体来说，就是IntentionAgent先识别用户意图（是想查询、规划还是咨询政策），然后调用对应的ItineraryPlanningAgent生成行程。这个初版系统能够处理"我想去杭州玩三天"这样的简单需求，生成包含景点、酒店、餐饮的基础行程方案。但很快我们发现了几个明显的问题：首先，用户体验很不智能，每次都要重复说自己的偏好，基础信息，系统没有记忆能力；其次，当用户问一些平台政策类问题，比如"平台的优惠政策"，纯粹靠大模型容易产生幻觉；第三，意图识别比较简单，采用关键词匹配的方式，准确率只有65%，复杂意图识别不出来。

针对这些问题，我们进行了第一轮优化。我设计了一套两层记忆系统来解决用户偏好记忆的问题。短期记忆基于滑动窗口机制，保存当前会话的最近10轮对话用于理解上下文；长期记忆则采用PostgreSQL关系型数据库进行持久化存储，保存用户偏好（酒店品牌、航空公司、座位偏好等）、历史行程记录、完整聊天历史，实现跨会话的数据共享。同时我还加入了一个PreferenceAgent，基于大模型的语义理解能力，它能够智能识别用户是在追加偏好（比如"我还喜欢如家"）还是覆盖偏好（比如"我搬家到上海了"），让系统真正能记住用户的习惯。这个改进上线后，用户偏好记忆准确率达到了95%左右，基本不需要重复输入了。同时我还引入了异步LLM总结机制，定期对历史对话和行程进行自动摘要，进一步提升了记忆的可用性。

后来为了解决知识支撑不足的问题，我们又做了第二轮优化，实现了一个RAG知识库系统。我采用了本地化部署的方案：用Milvus作为向量数据库，BGE-m3做Embedding模型，把公司内的差旅相关文档（优惠政策、城市攻略、常见问题FAQ等）都做了向量化存储。在文档处理环节实现了Chunking策略，针对长文档采用滑动窗口切分，保证检索精度。当用户咨询政策类问题时，系统会先通过余弦相似度检索相关文档片段，再让大模型基于检索内容生成回答，同时返回文档来源实现可追溯性，大大增加了可信度。另外，我们还发现用户经常需要查询实时信息，比如"杭州明天天气怎么样"、"北京明天的限行政策"这类问题。针对这个需求，我加入了InformationQueryAgent，集成了Google搜索引擎

的API，能够联网获取实时信息并用大模型自动摘要。这样系统既能基于知识库回答规范性问题（如差旅政策、报销标准等静态知识），又能处理实时性查询（如天气、景点实况等动态信息），功能更完善了。

接下来我们重构了意图识别和Agent调度模块，这是第三轮优化的重点。最初的IntentionAgent只是简单的关键词匹配，我做了一轮测试集评估，收集了1000个真实用户输入，发现准确率只有65%。主要问题在于无法处理同义表达、复合意图和上下文依赖。于是我们把意图识别升级成基于语义理解的多意图识别系统，能够识别6大类意图：规划行程（itinerary_planning）、查询记忆

（memory_query）、管理偏好（preference）、知识问答（rag_knowledge）、信息查询（information_query）、实时检索。同时我还实现了一套动态的Agent调度机制，采用"优先级+并行"的混合模式。具体来说，根据意图识别结果动态分配优先级，同优先级的Agent可以并行执行，不同优先级按依赖关系串行执行。比如用户说"我想3月11日从北京去杭州出差一周"，系统会并行调用EventCollectionAgent（提取结构化信息）和MemoryQueryAgent（查询历史偏好），然后再调用ItineraryPlanningAgent整合生成行程。这个并行调度机制让整个流程从串行的30秒优化到15秒，响应速度快了一倍，意图识别准确率也提升到了90%以上。

在工程实现上，我参考了Claude Code的设计思路，把所有子Agent重构成独立的Skill插件。这样做的好处是新增功能非常简单，每个Skill可以独立开发、测试和部署，并且支持懒加载机制，未使用的Skill不会被加载进内存，项目启动速度优化到了3秒以内。其次，我引入了LazyAgentRegistry动态发现机制，系统启动时自动扫描.claude/skills目录，发现并注册所有可用的Agent插件，无需手动配置。最后，我引入了Redis作为缓存层来优化热数据访问性能，将短期记忆（会话状态）、用户偏好热数据、LLM总结结果都缓存到Redis中。这样做的好处是支持分布式部署时的会话共享（短期记忆TTL设置为1小时），减少PostgreSQL的高频查询压力，避免重复调用LLM进行历史总结。Redis与PostgreSQL形成了"缓存+持久化"的互补架构，显著提升了系统的并发能力和响应速度。

目前这个智能旅行助手系统已经能够支持完整的行程规划流程，覆盖意图识别、事项收集、偏好管理、知识问答、信息查询、行程生成这六大功能模块。整体来看，这套系统让普通用户可以像有专业旅行顾问一样，快速获取个性化的行程建议和全方位的旅行信息，大大降低了旅行规划的门槛。

项目问题

1. 简单说说AgentScope是什么框架

AgentScope是由阿里开源的生产就绪的多智能体开发框架，是一个"易于使用的Agent框架，具有适应大模型不断提升能力的核心抽象设计"。专为日益增强的大语言模型设计，充分利用模型的推理和工具使用能力，避免过度严格的提示词和僵化的编排方式。采用基于**消息传递（Message-Passing）**的架构，Agent之间通过Msg对象进行通信和协作。 **技术特点：**

- 支持Python和Java两种开发语言

- Async-first异步架构，支持高并发
- 内置ReMe记忆管理套件，支持跨Agent记忆共享
- 提供零代码工作站（Workstation）和代码两种开发方式
- 集成模型评估和强化学习能力

2. 为什么选择用AgentScope，不用LangGraph

AgentScope的特点：

- **基于消息传递的架构**：采用Actor模型的分布式框架，Agent之间通过Msg对象进行清晰的消息交互
- **开箱即用更丰富**：内置ReAct、记忆管理、人机协同、语音交互等功能，5分钟快速上手
- **多Agent协作更友好**：专注于多Agent系统，提供灵活的消息中心和调度机制
- **生产就绪**：实时控制、中断处理、沙箱执行等企业级能力完善，内置AgentStudio可视化工具

在这个项目里

1. 消息传递架构适合多Agent协作。AgentScope采用Actor模型，Agent之间通过Msg对象传递结构化数据。我们系统有6个子Agent，需要频繁传递意图识别结果、调度决策、上下文信息，消息传递的方式让每个Agent的输入输出都很清晰，调试时能直观看到整个数据流向。LangGraph是基于全局State的状态图架构，Agent数量多的时候状态管理容易变得臃肿。
2. AgentBase基类适合插件化架构。所有子Agent继承AgentBase，统一暴露reply方法。这样我们的懒加载注册器可以动态扫描目录、自动发现并实例化Agent插件，新增功能只需写一个继承AgentBase的类，无需修改核心代码。LangGraph的节点需要手动定义状态Schema和边的转换逻辑，做不到这种即插即用。
3. 异步支持适合并行调度。我们实现了"优先级+并行"的混合调度——同优先级Agent并行执行，不同优先级串行执行。AgentScope的异步reply接口配合asyncio实现并行非常自然，把响应时间从30秒降到了15秒。LangGraph的并行需要在图结构中预定义，运行时动态调整并行策略比较困难，而我们哪些Agent需要并行完全取决于意图识别的实时结果。

3. 简单介绍一下你设计的几个Agent

项目中共实现了6个核心Agent，分别是记忆查询Agent、事项收集Agent、偏好管理Agent、知识问答Agent、信息查询Agent和行程规划Agent：

1. **MemoryQueryAgent(记忆查询)**：基于用户的长期记忆回答历史相关问题。它能够查询用户旅行历史(trip_history)、用户偏好(preferences)和历史对话记录(chat_history),使用LLM基于记忆生成自然语言回答,适用于"我去过哪些地方""我上次去北京是什么时候"等场景。
2. **EventCollectionAgent（事项收集）**：负责从用户输入中提取结构化信息，如出发地、目的地、日期、时长等关键要素，为行程规划提供基础数据。

3. **PreferenceAgent (偏好管理)**：管理用户的旅行偏好，包括酒店品牌、航空公司、座位偏好等。它能够智能识别用户是在追加偏好（比如"我还喜欢如家"）还是覆盖偏好（比如"我搬家到上海了"），准确率达到95%。
4. **RAGKnowledgeAgent (知识问答)**：基于Milvus 向量数据库和BGE中文向量模型，实现差旅政策、城市攻略等企业知识的智能检索和问答，准确率达到95%。
5. **InformationQueryAgent (信息查询)**：集成Google搜索引擎，让Agent具备联网查询能力，能够获取天气、景点实况等实时信息。
6. **ItineraryPlanningAgent (行程规划)**：整合前面所有Agent的结果，生成完整的个性化行程方案，包括景点推荐、酒店预订、交通安排等。

4. 这个项目的核心指标有哪些，每个指标是怎么算出来的

这个项目在性能、准确性和用户体验三个维度都有明确的量化指标，以下是核心指标及其计算方法（每行一个指标）：

- **意图识别准确率90%+**：在1000条测试集上，统计IntentionAgent识别正确的意图数量除以总样本数，从最初基于关键词匹配的65%提升到基于LLM语义理解的90%+
- **知识库问答准确率95%**：在RAG知识库的测试集中，统计RAGKnowledgeAgent返回的答案与标准答案一致的比例，通过混合检索（向量+BM25）和相似度阈值过滤实现
- **用户偏好记忆准确率95%**：统计PreferenceAgent正确识别追加/覆盖动作的成功率，在测试集中对比系统保存的偏好与用户实际意图的一致性
- **系统响应时间从30秒优化到15秒**：从用户发送query到返回完整结果的端到端耗时，通过引入优先级并行调度机制，让Priority 1的5个信息收集Agent并行执行而非串行，响应时间减半
- **用户偏好缓存命中率85%**：Redis缓存层的命中率，统计偏好查询时直接从Redis获取的次数除以总查询次数，避免高频访问PostgreSQL

介绍下1000条测试集

- **测试集来源**：从公司内部旅行系统日志中抽取真实用户查询,经过脱敏处理后筛选出1000条。
- **测试集内容**：覆盖5大类意图(行程规划、偏好管理、记忆查询、知识问答、信息查询),每类约200条。包含口语化表达("明天去杭州")、同义表达("订酒店"vs"订宾馆")、上下文依赖("那边天气怎么样")、复合意图("我要去北京,喜欢住汉庭")等情况。
- **测试集格式**：

代码块

```
1  {
2    "query": "我要3月11日从北京去上海,喜欢住汉庭",
3    "ground_truth": ["itinerary_planning", "preference"]
4  }
```


- **如何判断是否正确：**都是人工标注的

- **意图识别准确率：**对1000条测试集,逐条调用IntentionAgent,提取输出JSON中的意图类型列表,与标注对比。完全匹配算对,否则算错。准确率 = 正确数 / 1000
- **偏好记忆准确率：**从1000条中筛选出约200条偏好相关的query(标注包含"preference"意图的),让系统实际执行完整流程,检查最终写入长期记忆的偏好数据是否正确(比如"我还喜欢如家"应该追加到hotel_brand列表,"我搬家到上海了"应该覆盖home_location字段)

5. 这个项目采用的是什么Agent架构，为什么选择这个架构

这个项目采用的是**Plan-and-Execute架构**，这是一种先规划再执行的两阶段架构模式。

Plan-and-Execute架构的核心思想是将Agent的工作分成两个阶段：Planning阶段由IntentionAgent负责理解用户意图并生成完整的执行计划，Execute阶段由OrchestrationAgent按照计划调度各个子Agent执行具体任务。这种架构的优势在于规划和执行解耦，IntentionAgent专注于理解和规划，OrchestrationAgent专注于调度和执行，职责清晰。

选择Plan-and-Execute而非ReAct的原因主要有三点：

第一是任务复杂度，我这个旅行助手项目需要协调多个子Agent（事项收集、偏好管理、知识问答、信息查询、行程规划等），如果用ReAct的Thought-Action-Observation循环，每次只能执行一个Action然后观察结果再决定下一步，效率太低。而Plan-and-Execute可以一次性规划好所有需要调用的Agent及其优先级，然后并行执行，大大提升了效率。

第二是成本控制，ReAct架构需要多轮调用LLM进行推理，每轮都要消耗token，而Plan-and-Execute只需要IntentionAgent调用一次LLM生成计划，后续执行不需要反复调用LLM做决策，token消耗更少。

第三是可预测性，ReAct的循环次数不确定，可能陷入死循环或者过早终止，而Plan-and-Execute的执行流程是确定的，IntentionAgent生成的计划明确指定了要调用哪些Agent、调用顺序和优先级，执行过程可控可预测。

6. 意图识别Agent的核心逻辑是什么

IntentionAgent是整个系统的核心，负责理解用户输入并生成Agent调度计划。核心逻辑包括四个步骤：

第1步：推理过程生成 - 分析用户query的核心诉求，识别关键实体和意图信号，判断是否需要结合对话历史进行消歧。

第2步：多意图识别 - 识别所有可能的用户意图（可以是多个），为每个意图分配置信度（0-1之间），说明识别该意图的原因。

第3步：智能Query改写 - 识别口语化表达并进行标准化，补全省略的上下文信息，提取和重组关键信息。

第4步：构建结构化决策 - 基于识别的意图，决定调用哪些子Agent，说明调用顺序和优先级。最后调用LLM进行意图识别，要求输出严格的JSON格式，包含reasoning（推理过程）、intents（意图列

表)、rewritten_query (改写后的查询) 和agent_schedule (Agent调度计划, 指明要调用哪些Skill) 。

在实现上, 我构建了一个详细的Prompt, 包含当前时间、对话历史、可调度的子Agent信息等上下文。

这里我采用了插件化的Skill架构, 所有的子Agent都是作为独立的Skill插件实现的 (如ask-question、memory-query、plan-trip等), 存放在统一的skill目录下。

因此, IntentionAgent需要动态获知当前系统中有哪些可用的Skill, 以及每个Skill的触发条件和功能描述。我通过动态加载机制扫描技能目录, 读取每个Skill的元数据 (名称、描述、触发条件等), 然后将这些信息注入到Prompt中, 让LLM能够了解所有可用的Agent选项。

这样设计的好处是: 新增或删除Skill时, IntentionAgent会自动感知到变化, 无需手动修改Prompt。

同时我还实现了Progressive Disclosure (渐进式暴露) 机制, 在意图识别阶段只加载元数据, 不加载详细的执行指令, 避免Prompt过长。

7. 你刚才提到Progressive Disclosure, 详细说下它是什么

Progressive Disclosure (渐进式披露) 是一种信息架构设计模式, 核心思想是"按需加载, 分层披露信息", 避免一次性加载所有信息导致的复杂度和token开销。在这个项目中, 我参考了Claude Code的Agent Skills设计, 实现了三层信息结构:

- **第1层: 元数据层 (Metadata)** - 包含所有Skill的名称和简短描述, 在意图识别阶段加载。这让IntentionAgent能够了解所有可用的Agent, 但不会被详细指令淹没。
- **第2层: 指令层 (Instructions)** - 对应Skill配置文件中的详细指令, 在具体Agent执行时按需加载。例如RAGKnowledgeAgent执行时, 会动态读取RAG生成回复的要求, 注入到Prompt中。
- **第3层: 资源层 (Resources)** - 包含Reference文件、数据文件、代码实现, 在实际使用时才访问。这种设计的好处是: 首先节省token, 意图识别阶段只需要加载元数据, 不需要加载所有详细指令; 其次提升准确性, 每个Agent只看到自己需要的指令, 不会被无关信息干扰; 最后易于维护, 每个Skill的说明和指令都在自己的Skill配置文件中, 修改时不影响其他部分。在实现上, 系统会动态扫描技能目录, 解析每个Skill的Skill配置文件, 提取元数据构建Prompt, 实现了灵活的按需加载机制。

8. 你提到了优先级和并行调度, 详细说一下

这是系统性能优化的关键设计。在最初版本中, 所有Agent都是串行执行的, 比如意图识别→事项收集→偏好管理→行程规划。后来我重构了调度机制, 引入了"优先级+并行"的混合模式。

优先级分组:

- **Priority 1 (并行执行)** - 信息收集类Agent, 包括记忆抽取、事项收集、偏好管理、网络检索、RAG。这些Agent都是信息获取, 互不依赖, 可以并行执行。

- **Priority 2（依赖Priority 1）** - 行程规划类Agent，即它需要使用Priority 1收集的信息。实现的时候相同优先级的task被放进一个批次，然后并行执行同一批次的所有协程。这样，Priority 1的5个Agent可以同时并行调用，大大缩短了总体执行时间。实际测试中，响应时间从串行的30秒优化到了15秒，提升了一倍。

9. 调度Agent的核心职责是什么

调度Agent是整个多Agent系统的调度中枢，负责协调各个子Agent的执行。它的核心职责包括：

- 1. 解析意图识别结果** - 从IntentionAgent的输出中提取agent_schedule（调度计划），获取每个Agent的调用信息、优先级、调用原因等。
- 2. 准备上下文信息** - 构建统一的上下文，包括：
 - 意图识别的reasoning、intents、key_entities、rewritten_query
 - 从MemoryManager获取的短期记忆（最近10轮对话）
 - 从LongTermMemory获取的用户偏好
- 3. 按优先级调度Agent** - 按priority排序task，同优先级的并行执行，不同优先级顺序执行。执行单个Agent时，会构造统一格式的输入Msg。
- 4. 聚合执行结果** - 将所有Agent的执行结果聚合成统一的JSON结构，包含status、agents_executed、results等字段。
- 5. 更新记忆系统** - 将重要信息写入长期记忆：
 - 如果是PreferenceAgent，保存用户偏好到LongTermMemory
 - 如果是ItineraryPlanningAgent，保存行程历史到LongTermMemory
 - 支持append和replace两种偏好更新模式 整个流程清晰有序，每个子Agent只需关注自己的业务逻辑，不需要知道调度细节。

10. 多Agent之间如何完成通信

这个项目采用了AgentScope框架的消息传递机制，所有Agent之间通过标准化的Msg对象进行通信。

- **Msg对象的结构**包含三个核心字段：name表示发送者的Agent名称，content是消息内容通常是JSON格式的结构化数据，role标识角色类型如user或assistant。这种统一的消息格式让所有Agent都能理解彼此的输出。
- **调度Agent作为消息中枢**，负责在各个子Agent之间传递消息。具体流程是：首先调度Agent从IntentionAgent接收意图识别结果，然后为每个子Agent构造标准的输入Msg，这个Msg包含上下文信息、调用原因、期望输出、以及前序Agent的执行结果。子Agent处理完后返回一个新的Msg对象，调度Agent解析这个返回的Msg，提取结果并聚合到一起。

- **并行通信的实现：**采用异步机制，对于同优先级的多个Agent，调度Agent会同时向它们发送Msg，每个Agent独立处理自己的消息并返回结果，调度Agent统一等待所有协程完成后收集结果。这种设计的好处是Agent之间解耦，每个Agent只需要知道如何处理输入的Msg和返回输出的Msg，不需要知道其他Agent的存在。
- **上下文共享机制**确保所有Agent都能访问必要的信息。调度Agent在准备上下文时，会从记忆系统获取短期记忆和用户偏好，然后把这些信息打包到Msg的content字段中传递给子Agent。如果某个Agent需要知道前面Agent的执行结果，调度Agent会把之前所有Agent的输出也包含在Msg中，实现信息的流转和共享。这种基于消息传递的通信方式，相比传统的函数调用或共享状态，具有更好的可扩展性和容错性，符合分布式系统的设计原则。

11. 你刚才提到的记忆系统，详细说一下设计思路

记忆系统是实现用户偏好记忆的核心，我设计了一套两层记忆架构：短期记忆+长期记忆。 **短期记忆 (ShortTermMemory)：**

- 存储：内存中的列表，保存最近10轮对话
 - 生命周期：会话级，会话结束后清空
 - 用途：为当前对话提供上下文，让Agent理解用户的连续提问
- 长期记忆 (LongTermMemory)：**
- 存储：PostgreSQL数据库持久化，存储用户偏好、历史行程、全量聊天记录
 - 生命周期：跨会话，永久保存
 - 用途：记住用户的长期偏好和历史行为，实现个性化推荐
- 异步LLM总结：**为了避免长期记忆过多导致token爆炸，我引入了异步LLM总结机制。在每次对话前，调用异步总结方法对历史聊天和行程进行总结，生成200字以内的摘要，作为system消息传给IntentionAgent。

12. 为什么选择PostgreSQL而不是MySQL

在做技术选型时，我对比了PostgreSQL和MySQL这两个主流关系型数据库，最终选择PostgreSQL主要基于以下考虑：

PostgreSQL的核心特点：

1. **对象-关系型数据库：**PostgreSQL不仅是关系型数据库，还支持面向对象的特性，如继承、自定义类型等，扩展性更强
2. **严格遵循SQL标准：**对SQL标准的支持更完整，包括复杂查询、窗口函数、CTE（公共表表达式）等高级特性
3. **MVCC并发控制：**采用多版本并发控制，读写操作互不阻塞，并发性能优异
4. **丰富的数据类型：**原生支持JSON、JSONB、数组、hstore、UUID等复杂数据类型，非常适合存储半结构化数据

相比MySQL的优势：

1. **复杂查询性能更好**：PostgreSQL的查询优化器更先进，对于多表JOIN、子查询、聚合分析等复杂场景，性能明显优于MySQL
2. **JSON支持更强大**：PostgreSQL的JSONB类型支持索引和高效查询，可以像NoSQL数据库一样操作JSON数据，而MySQL的JSON支持相对较弱
3. **并发写入能力更强**：得益于MVCC机制，PostgreSQL在高并发写入场景下不会出现锁竞争，而MySQL的InnoDB引擎在高并发时容易出现锁等待
4. **数据一致性保证更好**：PostgreSQL在故障恢复、数据完整性约束方面更严格，更适合对数据准确性要求高的场景

13. 你提到还引入了Redis缓存，详细说一下

我设计了一个"Redis缓存+PostgreSQL持久化"的互补架构：**Redis缓存的三类数据**：

1. **短期记忆（会话状态）** - TTL设置为1小时，支持分布式部署时的会话共享。
2. **用户偏好热数据** - 缓存用户的常用偏好（酒店品牌、航空公司等），减少PostgreSQL的高频查询。
3. **LLM总结结果** - 对长期记忆的LLM总结结果进行缓存，避免重复调用LLM。Key格式：user:{user_id}: summary, TTL设置为30分钟

缓存更新策略：

- **Write-Through**：写入时同时更新Redis和PostgreSQL，保证数据一致性
- **数据Lazy Loading**：读取时先查Redis，未命中再查PostgreSQL并写入Redis
- **TTL过期**：短期记忆1小时过期，偏好和总结根据更新频率设置不同TTL
- **性能提升**：并发能力从100 QPS提升到500+ QPS

14. 插件化的Skill Plugins架构是怎么实现的

我参考了Claude Code的设计思路，实现了一套完整的插件化架构，让新增Agent变得非常简单。

LazyAgentRegistry（懒加载注册器）：

工作机制分2步：

1. **自动发现** - 系统启动时扫描.claude/skills目录，找到所有Skill的位置，但这时候只是记录下"这个Skill在哪"，并不真的加载它
2. **懒加载（按需加载）** - 只有当某个Agent真正被调用时，才去加载这个Agent。比如用户问了知识库相关问题，IntentionAgent决定调用RAGKnowledgeAgent，这时候才去加载RAG相关的代码，加载后会缓存起来，下次就不用重复加载了。这样系统启动很快，不用一次性把所有Agent都加载进来

这种设计的好处：

- **零配置**：不需要手动注册Agent，系统自动发现
- **热插拔**：新增或删除Skill不影响核心代码

- **独立开发**：每个Skill都是自包含的，可以独立开发和测试
- **快速启动**：懒加载机制让系统启动速度优化到3秒以内

15. RAG知识库是怎么实现的

RAG（检索增强生成）知识库是RAGKnowledgeAgent的核心功能。

知识库中存储的文档包括**优惠政策**（如差旅报销标准、折扣规则）、**城市攻略**（如+景点介绍、交通指南、美食推荐）、**常见问题FAQ**（如订票流程、退改签政策、会员权益说明）等采用BGE-m3作为Embedding模型，采用混合索引（向量检索+关键词匹配）检索top3相关的chunk，将检索到的内容填入到prompt里交给大模型生成回复。

针对长文档采用滑动窗口切分，每个chunk 512 tokens，overlap 50 tokens，为每个chunk添加metadata（类别、来源文档等），保证检索精度和上下文完整性。

知识库问答准确率达到95%，基本能够准确回答差旅政策、报销标准等企业知识问题。

16. 向量数据库为什么用Milvus

选择Milvus的原因：

1. **分布式设计**：原生分布式设计，支持水平扩展（分片、副本），可从单节点平滑扩展到大规模集群，处理 PB 级向量数据；支持动态扩缩容（无需停机），可根据数据量 / 检索量灵活调整节点数量，适配业务流量波动；分片策略灵活（按向量维度、数据量、时间范围分片），满足数据持续增长的需求。
2. **性能优秀**：Milvus是专为高维向量搜索设计的数据库，支持多种索引算法（FLAT、IVF、HNSW 等）以及多种距离度量方式（余弦相似度 Cosine、欧氏距离 L2、内积 IP、汉明距离等），检索性能优于传统数据库的向量插件，亿级高维向量毫秒级召回。
3. **向量 + 标量混合检索**：支持过滤条件 + 相似度联合查询，更贴合业务。可先通过标量过滤缩小检索范围，再做向量相似性检索，避免全量向量检索的性能损耗，大幅提升精准度和检索速度，适配实际业务中“多条件筛选 + 相似性匹配”的核心需求。
4. **检索方式多样**：支持渐进式检索（先返回近似结果，再优化精度），适配低延迟高吞吐的业务需求（如实时推荐、语义搜索）。

17. 召回用的是什么检索算法

项目中采用的是**混合检索**（Hybrid Search），结合了向量检索和关键词匹配两种方式，互补优势，提升召回准确率。

1. **向量检索（语义匹配）**：使用**余弦相似度**作为相似度算法。用户问题通过Embedding模型转化为向量后，计算与知识库中所有文档片段向量的余弦相似度。余弦相似度只关注向量的方向，不考虑长度，非常适合文本语义匹配。优势是能够理解语义，即使用户换一种说法，只要意思相近就能召回
2. **关键词匹配（精确匹配）**：使用**BM25算法**进行关键词匹配。BM25是一种基于词频和文档频率的排序算法，是搜索引擎常用的经典算法。它会考虑关键词在文档中出现的频率，以及关键词的稀有程度

(IDF)。优势是对关键词精确匹配很敏感，比如用户问"报销标准"，包含"报销标准"这个词的文档会被优先召回

3. 混合检索流程：

- a. 对用户问题同时进行向量检索和BM25关键词检索
- b. 向量检索召回top_k=10个语义相关的文档片段
- c. BM25检索召回top_k=10个关键词匹配的文档片段
- d. 对两路召回结果进行融合排序（Reciprocal Rank Fusion），最终取top_3返回

18. 意图识别准确率从65%提升到90%+，是怎么做到的

主要通过三个方面的改进：

1. 从关键词匹配升级到语义理解：最初的IntentionAgent只是简单的关键词匹配，例如看到"去"就认为是行程规划，看到"喜欢"就认为是偏好管理。但这种方式无法处理同义表达和复杂意图。我重构后采用基于LLM的语义理解（在意图识别模块），构建了详细的Prompt，包含6大类意图的详细定义和触发条件。

2. 引入多意图识别：支持一个用户输入同时包含多个意图，例如"我想从北京去上海，喜欢住汉庭"同时包含行程规划和偏好管理。每个意图都有confidence置信度，IntentionAgent会根据置信度决定是否调用对应的Agent。

3. 利用对话历史消歧：引入短期记忆和长期记忆总结，让IntentionAgent能够理解上下文。例如：

- "我去过北京吗?" → memory_query（询问自己的历史）
- "北京南站的酒店平均多少一晚?" → information_query（询问客观信息）

19. 你的对话Agent如何处理超出上下文窗口的长对话

在当前实现中主要通过两层记忆+LLM总结来处理：

1. 滑动窗口机制：

- ShortTermMemory使用双端队列实现固定长度的滑动窗口
- max_turns=10，只保留最近10轮对话
- 当对话轮数超过10时，自动丢弃最早的对话

2. LLM异步总结：

- 在每次对话前，调用异步总结方法
- 对长期聊天历史（排除当前会话）和行程历史进行总结
- 使用LLM生成200字以内的摘要，作为system消息传给IntentionAgent
- 总结结果缓存到Redis（TTL 30分钟），避免重复调用LLM

3. 分层记忆管理：

- 短期记忆提供当前会话的连贯性
- 长期记忆总结提供跨会话的上下文
- IntentionAgent接收的消息列表包含：system（长期记忆总结）+ user/assistant（最近10轮对话）

20. 你做这个项目遇到过什么困难/挑战

主要遇到了三个方面的困难：

1. Agent调度的复杂性： 最初实现串行调度时，逻辑比较简单。但引入优先级和并行调度后，需要处理很多边界情况：

- 如何保证同优先级Agent真正并行执行（使用异步并行机制）
- 如何区分界定意图之间的区别 解决这个问题花了不少时间，最后通过仔细设计调度Agent的调度逻辑（在调度模块），实现了清晰的优先级分组和并行执行机制。

2. 记忆系统的设计权衡： 两层记忆系统在设计时面临很多选择：

- 短期记忆保留多少轮对话（最终选择10轮）
- 长期记忆是否需要LLM总结（引入了异步总结）
- 如何处理长期记忆的token爆炸（通过总结+截断解决）
- 用户偏好的追加和覆盖逻辑（PreferenceAgent的智能识别） 这些决策需要在性能、准确性、成本之间权衡。最终通过多轮迭代，找到了一个比较好的平衡点。

3. 插件化架构的实现：

- 动态模块加载（动态导入）
- 依赖注入（检查构造函数签名）
- 路径管理（确保Agent能导入工具类） 最终实现了一个清晰优雅的插件系统，新增Agent非常简单。

21. 如果你的Agent执行失败了，系统会怎么处理

我设计了多层容错机制来保证系统的鲁棒性。

首先在Agent级别，每个Agent的执行都有异常捕获，如果某个Agent执行失败，会记录错误信息并返回失败状态，但不会中断整个流程。

其次在并行执行时，我使用了特殊的异常处理机制，即使某个Agent抛出异常，其他并行执行的Agent也会继续运行，不会互相影响。

第三是重试机制，对于那些可重试的异常，比如网络超时、速率限制、服务器临时故障这些，系统会使用指数退避策略自动重试，最多重试3次，这样能应对临时的网络抖动。

第四是部分失败处理，如果多个Agent并行执行时有的成功有的失败，系统不会全部丢弃，而是会标记为部分成功，把成功的结果返回给用户，让用户至少能看到一部分信息。

这样设计的好处是系统不会因为单个Agent的问题就完全崩溃，而是优雅降级，尽可能提供部分功能，用户体验会好很多。

22. 你提到了重试机制，为什么不是所有异常都重试

重试机制需要谨慎设计，不是所有异常都应该重试。

我的判断标准是看这个错误重试后有没有可能成功。

比如网络超时、速率限制、服务器临时故障这些，都是临时性的问题，等一会儿重试就可能成功，所以要重试。但如果是认证失败、请求格式错误、业务逻辑错误这种，重试多少次都不会成功，就不应该浪费资源去重试。另外重试也不是立即重试，而是采用指数退避策略，第一次等1秒，第二次等2秒，第三次等4秒，最多等30秒。这样做有两个好处：一是给下游服务恢复的时间，二是避免大量请求同时重试反而加剧服务压力。最多重试3次，如果还失败就直接返回错误，这样既保证了系统可靠性，又避免了无效重试带来的资源浪费。

23. 你的RAG系统如何保证不会产生幻觉

大模型幻觉是AI应用的核心问题，我通过四层防护机制来控制。

第一层是Prompt强约束，在RAG Agent的配置中明确规定必须严格基于知识库内容回答，不得编造信息，如果知识库中没有相关信息就明确告诉用户找不到。

第二层是RAG知识增强，先通过向量检索找到相关的文档片段，然后把这些片段明确标记为知识库信息传给大模型，要求它仅基于这些内容生成答案。

第三层是相似度过滤，只使用top3最相关的文档，如果相似度过低就判定为无相关知识，直接返回找不到信息，而不是让大模型瞎猜。

第四层是文档溯源，返回结果时会附带用于生成答案的原始文档片段，用户可以查看和验证答案的准确性，如果发现问题可以追溯到具体文档进行优化。通过这四层机制，RAG知识库问答的准确率达到95%，把幻觉的程度控制在了可接受的范围内。

24. 你的系统如何处理用户的模糊时间表达

这是IntentionAgent的一个重要功能。用户经常说"明天去杭州"、"下周二出发"、"2月28日开始"这种模糊的时间表达，如果不处理，后续的Agent就无法准确理解具体日期。

我的解决方案是在IntentionAgent的Prompt中注入当前时间信息，包括年月日时分和星期几，然后明确告诉大模型"当用户说明天或者相对时间时，请根据当前时间推断出完整的日期"。这样IntentionAgent在分析用户意图时，就会自动把"明天"转换成具体的日期比如"2026年3月12日"，把"2月28日"补全为标准格式"2026-02-28"。后续的EventCollectionAgent和ItineraryPlanningAgent直接使用这些标准化的日期，就不需要再做时间解析了。

这种设计让用户可以自然地使用口语化的时间表达，系统能够准确理解并转换为标准格式。

25. 你的Skill Plugins架构和MCP有什么区别

1. 本质不同

- Agent Skills: Agent 具备的**具体执行能力/工具**（如搜索、计算、查数据库、调用API）。
- MCP: 一套**标准化通信协议**，定义模型/Agent 如何与外部工具、环境、数据交互。

2. 层级不同

- Skills: **功能层**，是“能做什么”。
- MCP: **交互层/协议层**，是“怎么规范地调用”。

3. 作用不同

- Skills: 提供实际业务能力，完成具体任务。
- MCP: 统一交互格式，让不同Agent、工具、服务能**安全、标准化、跨平台互通**。

4. 关系 MCP 是调用规则，Agent Skills 是被规则管理和调用的能力集合。

26. 你的系统如何保证高并发性能

系统的高并发性能主要通过五个方面来保证。

第一是异步架构，整个系统基于异步机制实现，所有Agent都采用异步方法，这样在等待LLM响应时可以处理其他请求，充分利用IO等待时间。

第二是并行Agent调度，通过异步并行机制让同优先级的Agent同时执行，比如5个信息收集类的Agent可以并行调用，大大提升了吞吐量。

第三是Redis缓存层，缓存了短期记忆、用户偏好热数据和LLM总结结果，减少了数据库查询压力，用户偏好的缓存命中率达到85%。

第四是FastAPI框架，它是基于Starlette的异步框架，天生支持高并发，可以自动处理请求并发。

第五是懒加载机制，Agent只在首次访问时才加载，系统启动速度优化到3秒，减少了冷启动开销。

通过这些优化，系统的并发能力从100 QPS提升到了500+ QPS。

27. 未来优化（必准备）

结合项目实践经验，我认为主要有以下几个大方向的改进空间。

- **完善全链路可观测性**：现在如果系统出问题了，很难快速定位是哪个Agent哪个环节出了问题。未来可以建立分布式追踪体系，记录每个Agent的调用链路、执行时间、token消耗，从Trace级别追踪单次请求，从Session级别分析用户会话，从User级别统计用户行为。这些数据可以和测试评估平台打通，帮助快速定位性能瓶颈，发现异常并告警。
- **优化意图识别机制**：目前所有用户输入都要调用LLM来识别意图，响应时间长且成本高。未来可以增加一个规则引擎，对于那些明确的高频意图比如"我要去北京""查天气"，直接用规则匹配秒级响应，不需要调用LLM。只有那些复杂的、模糊的意图才走LLM深度推理。这样既保证了准确率，又大幅提升了响应速度，降低了成本。

- **增强Agent协作模式：**目前的调度逻辑相对固定，未来可以引入更灵活的Handoffs机制，让Agent之间能够互相转交任务，比如行程规划Agent发现需要更多信息时，可以主动转交给事项收集Agent补充。同时引入动态Routing，主规划Agent根据任务的复杂度和优先级，智能决定调用哪些子Agent、调用顺序和并行策略，实现更智能的任务编排。
- **流式响应和思维链可视化：**现在用户提问后要等所有Agent执行完才能看到结果，用户体验不好。未来可以实现流式输出，Agent执行完一个就立即返回部分结果，同时把Agent的推理过程可视化展示出来，让用户看到系统正在"思考"什么、调用了哪些Agent、每步得到了什么结果，既提升了用户体验，也增强了系统的可解释性。
- **多模态能力扩展：**目前只支持文本输入，未来可以支持图片和语音输入。比如用户上传一张酒店照片，系统能够识别酒店类型和环境，理解用户的偏好；或者用户拍一张景点照片，系统能够识别景点并推荐相关行程。通过接入多模态大模型，让Agent能够理解和分析视觉、听觉信息，大大拓展系统的应用场景。

28. 这个项目的分工情况是怎样的

这个项目是在我们业务组内部的一个创新项目，当时是3个人一起做的，我主要负责核心架构设计和Agent系统的实现。

- **我的工作**主要集中在三个方面。第一是整体架构设计，包括Plan-and-Execute架构的选型、多Agent协作机制的设计、记忆系统的分层架构等。第二是核心Agent的开发，我负责实现了IntentionAgent意图识别、OrchestrationAgent调度协调、PreferenceAgent偏好管理这三个核心模块，还有插件化的Skill架构和LazyAgentRegistry懒加载机制。第三是性能优化，我做了并行调度、Redis缓存、异步架构等一系列优化工作，把响应时间从30秒优化到15秒，并发能力从100 QPS提升到500+ QPS。
- **另外两位同事的分工**是这样的。一位同事主要负责RAG知识库模块，包括Milvus向量数据库的搭建、BGE embedding模型的选型和调优、混合检索算法的实现，还有知识库文档的收集和预处理，他把知识库问答准确率做到了95%。另一位同事负责前端界面和后端API，他用FastAPI搭建了RESTful API服务，实现了用户认证、会话管理、流式响应等功能，同时还负责PostgreSQL数据库的schema设计和Redis缓存策略的实现。

八股问题

Skills

简述什么是 Agent Skills

Agent Skills 是由 Anthropic 提出并主导的**开放能力标准**，是面向 AI 编程工具（如 Claude Code、Cursor 等）的**模块化、可复用能力封装机制**。

它通过**标准化文件结构**，将领域知识、定制化工作流与可执行逻辑注入 LLM，让模型在不修改核心架构的前提下，精准、稳定地完成复杂任务。

它将业务逻辑、SOP、工程最佳实践与可执行脚本统一封装在结构化目录中，通常以 `SKILL.md` 作为核心入口文件。

有三个特点：

- **按需加载（渐进式上下文披露）**：Skills 采用**延迟加载、按需激活**的机制，不会一次性全量注入上下文。Agent 仅在任务匹配时才加载对应技能内容，有效降低 Token 开销，减少上下文干扰，提升模型决策效率。
- **即插即用**：它是轻量级的技能插件，可以在不同的 Agent 之间复用，无需修改 Agent 的核心代码。
- **封装经验**：Skills 不仅是工具调用的封装，更重要的是**将专家经验、执行范式与完整解决方案结构化**。它让 Agent 掌握“如何系统性完成任务”，实现**工具使用到流程执行**的能力升级。

Agent Skills 与 MCP 协议的区别

MCP和Agent Skills虽然都在增强AI能力，但定位、解决的问题不同：

- **MCP是AI Agent 与外部系统的标准化交互接口层**，它负责帮AI连接外部世界——解决的是Agent去哪拿数据、能调用哪些系统接口的问题，比如让Agent连数据库、GitHub、Slack这些外部工具。
- **Agent Skills是行为规范层**，它负责帮AI固化专业流程——解决的是Agent如何正确执行任务的问题，教Agent按特定步骤、格式、标准完成工作。

实际最佳实践是：Agent加载一个Skill（比如故障排查技能），这个Skill会指挥Agent去调用通过MCP连接的日志系统工具、数据库工具，最终完成工作。

Skills 是如何通过渐进式披露节省 Token 的？

Agent Skills 通过 **渐进式披露（Progressive Disclosure）** 机制，将原本庞大的 Prompt 和工具定义从内存（Context）转移到了硬盘（File System）中。它不再一次性将所有能力塞入大模型的上下文窗口，而是像人类查阅资料一样，分阶段、按需读取信息。

这种机制通过以下 **三个层级** 来大幅节省 Token：

1. 第一层：元数据层

Agent 启动时，仅加载 Skill 的 name和 description，Agent 的系统提示词中只包含一份简短的技能目录。

2. 第二层：指令层

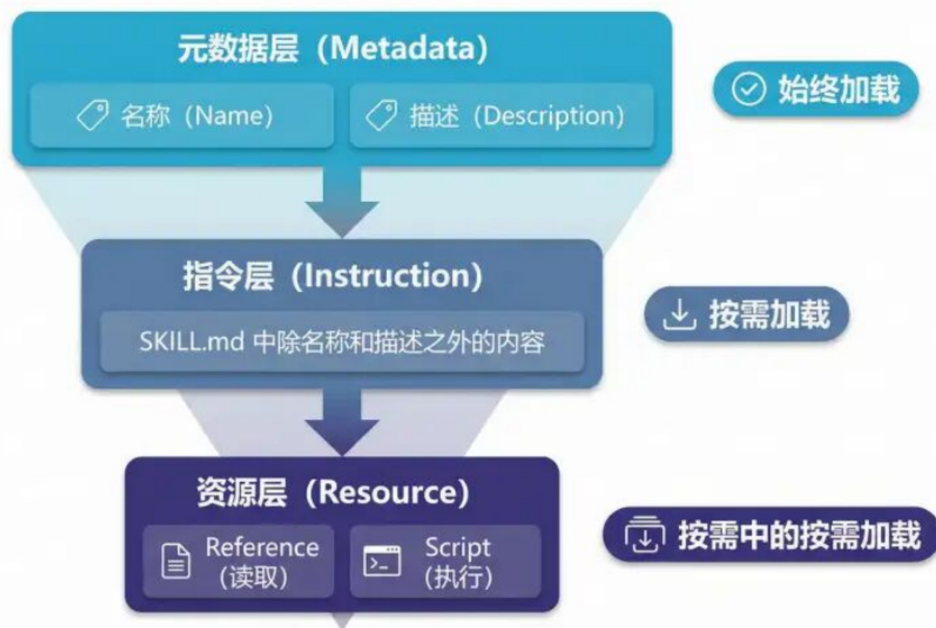
当用户的请求命中了某个 Skill 的描述时，Agent 会主动读取该 Skill 文件夹下的核心说明书 SKILL.md。只有在确定要使用该技能时，具体的SOP、操作步骤和注意事项才会被读取进上下文。

3. 第三层：资源与代码的外部化

这是 Skills 最节省 Token 的部分，分为两类情况：

- 静态资源reference：
 - 如果 Skill 包含大量的参考文档、模板或数据库 Schema，这些文件默认完全不进入上下文。
 - 只有当 SKILL.md 中的指令明确要求时，Agent 才会去读取特定文件的内容。
- 代码脚本script：
 - Skill 文件夹中通常包含 Python 或 Bash 脚本。
 - Agent 通过命令行工具直接运行这些脚本，脚本的源代码永远不会进入上下文，Agent 只能看到脚本运行后的输出结果。这比让 LLM 自己生成代码或阅读长代码要节省大量 Token。

Agent Skill 渐进式披露 (Progressive Disclosure)



指路：

Anthropic 官方文档

| <https://docs.anthropic.com/en/docs/agents-and-tools/agent-skills/overview>

Agent Skills 开放标准

| <https://agentskills.io>

ReAct

ReAct的任务解决轨迹是**Thought-Action-Observation**，可以简化为模型按照**Reasoning-Acting**框架。**Reasoning**包括了对当前环境和状态的观察，并生成推理轨迹。这使模型能够诱导、跟踪和更新操

作计划，甚至处理异常情况。ReAct的每一个推理过程都会被详细记录在案，这也改善大模型解决问题时的可解释性和可信度；Acting在于指导大模型采取下一步的行动，比如与外部源（如知识库或环境）进行交互并且收集信息，或者给出最终答案。

Agent包含哪些模块，实现了什么功能

Agent 就像一个多功能的接口，它能够接触并使用一套工具。根据用户的输入，Agent会规划出一条解决用户问题的路线，决定其中需要调用哪些工具，并调用这些工具。Agent = 大语言模型+规划+记忆+工具使用，具备以下关键能力：

- **规划（Planning）**：最核心最关键的部分，负责拆解复杂任务为可执行的子任务，并规划执行任务的流程。同时Agent还会对任务执行的过程进行思考和反思，决定是否继续执行任务，并改进决策策略。
 - **任务分解**：将复杂任务分解为可执行的子任务，让大模型逐步解决，例如将订外卖分解为选择餐厅+选择菜品两步。关键技术例如CoT、LLM+P等。
 - **反思**：Agent 通过完善过去的行动决策和纠正以前的错误来不断改进。关键技术例如React、Reflexion等。
- **记忆（Memory）**：包括短期记忆和长期记忆，用于存储会话上下文和业务数据等信息，来优化未来行为。
 - **短时记忆**：即上下文学习，由于受到Transformer上下文窗口长度的限制，它是短暂的和有限的。
 - **长期记忆**：则可对应为外部的向量数据存储，Agent 可在查询时引用，并可通过快速检索进行访问。
- **工具使用（Tools）**：通过调用外部工具（如API、插件）扩展Agent的能力，如文档解析、代码编译等。

RAG的流程

- 简略版

RAG一般分为三步：首先将文本分割成块；然后使用embedding编码模型将这些块嵌入到向量中，将所有这些向量放入索引中；最后为LLM创建一个提示，告诉模型根据我们在搜索步骤中找到的上下文来回答用户的查询。

在运行时，使用相同的embedding编码器模型将用户查询向量化，然后针对索引执行该查询向量的搜索，找到前k个结果，从向量数据库中检索相应的文本块，并将他们作为上下文输入到LLM提示中，最后总结输出答案。

- 复杂版

Indexing: 建库

- **数据收集与处理**：系统从各种数据源（如文本文件、PDF、网站、数据库或API）中收集数据，并转换成统一的纯文本格式。
- **数据分割**：将处理后的文本分割成适当大小的块，以便于后续的检索和管理，常用分割方法有固定长度分块、重叠分块等
- **向量化表示**：使用预训练的模型（如BERT、BGE等）将文本块转换为向量表示，这些向量捕获了文本的语义信息。
- **索引构建**：将这些向量存储在专门的数据库中，构建索引结构（如倒排索引或向量索引），以便快速检索。

Retrieval: 检索

- **查询编码**：当用户提交查询时，使用相同的编码器将查询转换为向量表示。
- **相似性计算**：计算查询向量与索引中的文档向量之间的相似度，常用的相似性度量方法包括余弦相似度和欧氏距离。
- **排序与选择**：根据相似度得分对文档进行排序，并选取排名最高的前K个文档或文档片段作为与查询最相关的文档。
- **查询优化（可选）**：进行重新排序或过滤，以提高检索结果的质量。

Generation

- **上下文整合**：将检索到的文档片段与用户的原始查询结合，形成一个连贯的提示（prompt），为生成模型提供丰富的上下文信息。
- **生成响应**：生成llm模型基于这些上下文信息和原始查询生成最终的回答。生成的文本不仅利用了检索到的信息，还结合了模型的语言生成能力，以确保回答的准确性和流畅性。
- **输出优化**：在生成阶段，可能会加入后处理步骤，如答案的置信度评估、多候选答案筛选、格式解析等，以确保生成的答案是相关且准确的。

追问：实际项目中可以有哪些优化

召回源可以考虑多路召回，例如稀疏召回，语义召回，字面召回等。对于多路召回阶段和召回分数对齐的问题，还会采用在召回后面加一个重排序的阶段，精简召回数并提升召回的质量。另外，其中的embedding模型、rerank模型、以及生成模型，都可以根据系统回答的指标情况，进行针对性的微调。

追问：RAG一般怎么做效果评估

RAG做效果评估主要针对检索和生成两个环节。

对于检索环节，可以采用MRR即平均倒排率，前k项的Hits Rate命中率，NDCG排序指标等。

对于生成环节，首先是量化指标，例如Rouge-L，文本相似度，关键词重合度等指标；第二个是多样性，除了准确度，还可以评估生成答案的多样性，看看模型是否能够生成多种合理且相关的答案；除此之外，还需要引入人类评估，进行人工评估，一般是负责该项目的产品经理和测试人员，以及内测的普通用户对模型的回答进行质量、准确性、连贯性的评分；另外，还应该考虑资源效率，尤其是在资源受限的环境中，看看RAG是否能够以合理硬件资源效果提供更好的性能。

使用RAG能解决哪些问题

- **提升生成内容的准确性：**通过引入外部知识库的信息，能够在生成文本时补充模型自身的知识缺陷，提高生成内容的准确性和可靠性。尤其是在特定领域中，RAG可能检索到相比模型训练时更丰富的知识。
- **即时更新知识：**RAG模型具备检索库的更新机制，可以实现知识的即时更新，无需重新训练模型，从而提供与最新信息相关的回答。
- **增强可解释性：**由于RAG模型的答案直接来自检索库，其回复具有很强的可解释性，用户可以核实答案的准确性，从信息来源中获取支持。
- **缓解幻觉问题：**让大模型避免对不了解的内容进行胡说八道。

文档解析

文档解析的作用是什么，解析不好会造成什么影响？

在RAG体系里，文档解析是将原始文档转换为可检索的内容的第一步，对后续的向量检索和生成式模型至关重要。它的核心价值在于：

1. **提取关键信息：**准确地从文档中获取可用文本、结构化数据以及上下文关系。
2. **保留文档结构：**对章节层次、表格、列表等结构的保留能帮助检索阶段更精准地定位答案。
3. **保证文本质量：**减少OCR或编码格式导致的噪音与错误，从而提高检索召回的精准度。

如果解析不当，容易导致：

- **召回错误或缺失：**信息被遗漏或文本打乱后，检索阶段就很难拿到正确答案。
- **上下文扭曲：**段落或跨页内容被拆分错位，模型生成时会出现无关或错误的回答。
- **系统不稳定：**一些质量很差的解析结果可能在下游产生连锁反应，导致用户体验大幅下降。

给你一些不同格式的文档，文档解析的大致思路？

1. **格式识别与路由：**首先根据文件类型（PDF、DOCX、图片、HTML等）将文档路由给不同的解析子模块。
2. **文本抽取：**
 - **PDF/Word：**使用现成的解析库或自研工具来获取文本内容和粗略的排版信息。

- **扫描版PDF/图片**：通过OCR引擎识别文本，并尽量恢复段落、表格或公式等结构。
- 3. **布局 and 结构化处理**：利用布局分析或者自然语言处理手段恢复文档的层级和排版，对表格、标题、列表等进行拆分或标注。
- 4. **切分与清洗**：在提取到的内容基础上进行去噪（比如去除页眉页脚、重复水印等），并按照自然段或逻辑单元做分块。
- 5. **元数据标注**：给每个切分后的片段添加文档名、章节标题、页码等元数据，方便后续检索和溯源。

怎么提升OCR的准确率？

1. **图像预处理**：在OCR之前进行去噪、增强对比度、纠偏等操作，减少因扫描质量造成的识别错误。
2. **分块识别**：针对文档中表格、公式、文本区域，采用分块方式单独识别并保留其结构，而不是一次性把整页识别完。
3. **OCR引擎选择**：结合场景特点（如语种、字体）选择适配性高的OCR模型，有些模型对手写体或模糊文字有更好的鲁棒性。
4. **字典和语言模型约束**：基于领域词库（例如医学术语、财会术语）对OCR输出做二次校正，修正常见拼写和字符混淆。
5. **多通道对比**：对重要文档，可能用不同OCR引擎并对结果进行对比或融合，尽量降低漏识别或误识别的概率。

多级文档怎么保证在检索阶段保留层级信息？

1. **结构化存储**：将文档各个章节、子章节的标题层级以及表格、图例的位置都转换成可索引的元数据，例如使用JSON、XML或者数据库的方式存储。
2. **Chunk附带上下文**：在每个Chunk的属性里记录其在文档中的位置（页码、章节索引）以及父子关系（上级标题、下级标题），必要时也可以记录所有祖先节点的名称。
3. **检索策略**：对查询做匹配时，可以先通过Chunk的文本向量相似度检索出候选结果，再根据层级信息来做二次过滤或排序。例如，如果用户想查“第二章里的某个概念”，那么就可以优先考虑第二章内的Chunks。
4. **可视化回溯**：当将答案呈现给用户时，可以用层级信息告诉用户“本段内容位于第3节第2小节”，让用户更好地理解上下文并找到原文位置。

怎么评价文档解析效果？

1. **OCR准确率**：统计从图片或扫描PDF中提取的文本与人工校对文本的差异（字符级或词级错误率）。

2. **段落完整度**：对自然段、列表或表格进行完整性对比，看是否有切分过度或拼接错误。
3. **结构保真度**：如果文档包含表格、分栏、多级标题等结构，评估解析后是否能够正确地保留和表示。
4. **检索测试**：在上线阶段，用一组真实查询对解析结果做向量检索，查看召回率和准确率是否符合预期；如果解析质量高，相关查询自然能得到更准确的Chunks。
5. **可溯源性**：是否能通过解析后的结果快速定位到文档原始位置（页码、段落），这也可以作为衡量解析质量的一个指标。

Chunking

为什么要做Chunking

- RAG需要先将长文档拆分成可检索的文本块（chunks）供向量检索或索引，以保证检索效率和模型处理的上下文长度限制。
- 不经过合理切分，会导致检索和生成时出现重要信息丢失或上下文拼接不完整等问题。
- 适当切分可以减轻检索负担，提高检索准确度和后续模型生成效果。

怎么保证跨页图片和表格不被拆分到不同的chunk中？

- 在解析阶段标记表格/图片的起始和结束位置，将它们视为不可拆分单元；
- 引入版面检测或分页信息（如PDF解析时的page number、坐标等）来判定表格或图片是否被跨页切断，然后在合并逻辑中优先“整块”保留。
- 当其Token大小超过预设阈值时，需将其单独放进一个Chunk，而不是与其他文本混合。

如何避免将一个完整段落或语义单元拆分到多个Chunk中？

- 通过句子边界检测、段落级ID或layout信息（如行距、换行检测）标识完整段落，确保同一段落内的语义单元尽可能保留在同一个Chunk。
- 引入Token计数逻辑时，先检查合并后的长度是否超过阈值。若不会超，则整段放在一个Chunk，否则在自然的语义边界（如句号、分号等）进行拆分。

如何根据chunk定位到原文位置？

- 在每个Chunk中记录其对应的页码、段落ID、或者其他排版信息（如“（p1）”、“paragraph_id=xxx”）；
- 通过结构化的元数据（JSON或数据库字段）与检索系统集成，能让RAG在检索结果中快速定位到原文位置；
- 这样不仅能在检索环节准确找到对应Chunk，也方便前端应用在输出时展示文档来源。

如何评价Chunk切分效果？

- **信息完整度**：表格/图片/标题等是否完整保留，量化为“完整保留率”或“信息缺失率”；
- **语义连贯度**：检查原文被过度割裂的比例，如“句子被打断”的次数占比；
- **段落/标题结构准确率**：切分后标题与对应正文保持正确层级的比例；
- **搜索检准率**：在RAG场景下，经切分后的文档块能否有效提升检索与回答准确率；

RAG检索

RAG怎么做检索

主要的检索算法有稀疏检索和密集检索两种：

- **稀疏检索**代表算法BM25，是一种基于关键词的检索算法，通过计算查询词与文档中的关键词匹配程度来评估文档的相关性。它使用TF-IDF（词频-逆文档频率）权重来衡量关键词的重要性。

- 词频（TF）：一个词在文档中出现的频率。一个词在文档中出现的次数越多，它就越重要：

$$TF(t, d) = \frac{\text{词汇}t\text{在文档}d\text{中出现的次数}}{\text{文档}d\text{中的总词数}}$$

- 逆文档频率（IDF）：反映了一个词在整个文档集中有多罕见。如果一个词在很多文档中都出现，那么它的重要性就低；反之，如果它在少数文档中出现，则它的重要性就高。

$$IDF(t) = \log\left(\frac{\text{文档总数}}{\text{包含词汇}t\text{的文档数量}}\right)$$

- TF-IDF： $TF - IDF(t, d) = TF(t, d) \times IDF(t)$

BM25的核心思想是计算查询词项和文档词项之间的相关性得分，然后综合这些得分来评估整个文档的相关性。BM25的优点在于其简单性和有效性，它能够快速计算文档与查询的相关性得分，并且通常在实际应用中表现良好。然而，BM25也有局限性，例如它不考虑词语的顺序和语义，也不考虑文档的长度对相关性的影响。

- **密集检索**：直接计算用户查询向量与文档之间的相似性，代表性指标有：

指标	公式	特点	应用场景
余弦相似度	$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\ \mathbf{A}\ \ \mathbf{B}\ }$	忽略向量长度，关注方向相似性。 范围：[-1, 1]，1表示完全相同。	文本相似度、推荐系统
欧氏距离	$d = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$	反映向量绝对距离，值越小越相似。对尺度敏感。	聚类分析、图像检索
曼哈顿距离	$d = \sum_{i=1}^n \ A_i - B_i\ $	计算速度快，对高维稀疏数据友好。	高维数据（如用户行为特征）
皮尔逊相关系数	$r = \frac{\text{cov}(\mathbf{A}, \mathbf{B})}{\sigma_A \sigma_B}$	衡量向量线性相关性，范围：[-1, 1]。对中心化数据敏感。	基因表达分析、用户评分预测
马氏距离	$d = \sqrt{(\mathbf{A} - \mathbf{B})^T \mathbf{S}^{-1} (\mathbf{A} - \mathbf{B})}$	考虑特征协方差，消除量纲影响。	分类任务中的异常检测

混合检索方案

- 意图识别与路由：**通过简单的规则或训练分类模型对查询进行分类。如果检测到查询是流程/制度类问题（通常包含“制度”、“流程”等关键词），则可以对BM25检索给予更高权重；如果是开放性思考类问题（包含“如何”、“怎么办”等），则侧重向量检索结果。此前置步骤保证不同查询走最合适的检索路径。
- BM25检索：**构建文档的关键词倒排索引（可使用Elasticsearch或其他搜索库），检索出Top N候选文档片段。BM25根据查询词在文档中的频率、文档长度等打分。对于短查询，可直接采用BM25结果；对于长查询，BM25结果可作为补充。
- 向量检索：**利用Embedding模型将查询编码成向量，在Milvus向量数据库中进行近邻搜索，获取Top N候选片段。向量检索能找出语义相关的内容，即使字面不匹配。
- 结果合并与去重：**将两种检索的候选列表合并。由于BM25分数和向量相似度分值不在同一量纲，需进行归一化处理。例如，可将BM25分数归一到0-1区间，向量相似度天然在0-1（如余弦相似度）。然后按一定策略融合，如线性加权组合或者直接取两者结果集的并集。在组合过程中处理重复文档（相同chunk多次出现）以避免干扰。
- 提高召回率：**混合检索确保潜在相关结果进入候选集。尤其在只用向量或只用BM25无法检索到某些答案时，另一种方式可以补充，使真正相关的片段不被漏掉。